JNEOPALLIUM · DEMO 06
TEST REPORT

Complete Test Report

Cybersecurity Temporal Threat Correlation — results, evidence, and an honest reading

---

What was tested, how it performed, and — stated plainly and up front — exactly what the results do and do not prove. Written so a non-specialist can read every number with confidence.

Document	Verification & Test Report
Product	Jneopallium Cybersecurity Module (Demo 06)
Model	cybersecurity-temporal-threat-correlator 1.0.0-reference-temporal
Test scope	Unit, behavioural, model-quality, scale
Result	All gates passed (reference corpus)
Author	Dmytro Rakovskyi — Kharkiv, Ukraine
Date	23 June 2026
License	BSD 3-Clause

Contents

1 Executive summary of results

The Demo 06 cybersecurity module was verified at four independent levels: **unit tests** (individual components behave correctly), **behavioural tests** (the running demo makes the right decisions), **model-quality metrics** (the trained correlator separates attacks from benign activity), and **scale evidence** (the training pipeline holds up at a 100 GiB logical corpus target). Every gate passed.

Test layer	What it checks	Result
Unit suite	35 component-level cases + interface invariant	**Pass** — all cases, no regressions
Behavioural	7 end-to-end demo assertions	**Pass** — all seven hold
Model quality	Precision / recall / F1 / false-positive rate	**1.0 / 1.0 / 1.0 / 0.0** on reference data
Scale	100 GiB logical-corpus target & guardrails	**Pass** — target reached, F1 gate held

The single most important sentence in this report

The perfect scores below are PIPELINE evidence on a deterministic reference corpus — they prove the system is wired correctly end to end. They are NOT a claim of real-world detection accuracy, which must be earned on external datasets. We state this here, at the top, on purpose.

2 What we tested, and why

A security product earns trust by being tested the way it will be used — not just "does the code run", but "does it make the right call, refuse the wrong call, and explain itself." Our verification is therefore layered, from the smallest part to the whole system:

- **Components in isolation** — does each neuron and processor do its one job correctly, even in awkward edge cases (a flood of traffic, a forbidden system-call sequence, a quarantine that must expire)?
- **The system as a whole** — when three realistic streams run together, does the demo raise the attack, calm the planned maintenance, surface the quiet leak, and never block anything?
- **The trained model** — given windows of events it has never seen, does it tell attacks from benign activity, and are its confidence scores honest?
- **The pipeline at scale** — does training stay correct, bounded, and reproducible when the logical corpus is pushed to 100 GiB?

3 Test environment

Item	Value
Platform	Jneopallium multi-module Maven project (master + worker)
Runtime	Java 17+ worker; Python 3 trainer (no third-party packages)
Model under test	cybersecurity-temporal-threat-correlator, 1.0.0-reference-temporal

Feature count	34 temporal-window features
Decision threshold	0.2 (auto-selected on validation)
Safety mode	ADVISORY
Reference command	train_temporal_model.py --reference-multiplier 1000 --target-corpus-bytes 100gb …
Determinism	Fully reproducible: same command → same checksum and metrics

4 A plain-language primer on the metrics

Before the numbers, here is what each one means — in everyday terms. Throughout, a "window" is a slice of time on one entity that the model judges as either *attack* or *benign*.

Metric	Plain-language meaning	Best
Precision	Of the windows it flagged as attacks, how many really were	1.0 (no false alarms)
Recall	Of the real attacks, how many it caught	1.0 (missed nothing)
F1	A single fair blend of precision and recall	1.0
Accuracy	Of all windows, how many it judged correctly	1.0
False-positive rate	How often it cried wolf on benign windows	0.0
Mean time to detection	How long into an attack before it raised the alarm	Low (near 0 ticks)

Why precision and recall both matter

A tool that flags everything has perfect recall but useless precision (constant false alarms). A tool that flags nothing has perfect precision but zero recall (it misses every attack). F1 keeps both honest at once — that is why it is the gate we enforce.

5 Layer 1 — Unit tests (the SecurityModuleTest suite)

The unit suite covers **35 cases** plus a structural invariant. Each case pins down one piece of behaviour so a future change cannot quietly break it. In plain terms, the suite proves that:

- Ingestion is **rate-limited** — a flood cannot overwhelm the stages behind it.
- Signature matching fires on a **known-bad pattern**, and a **forbidden system-call sequence** is caught.
- The **tolerance filter** drops known-good patterns, and the **hard gate** always blocks hard-allow entities and protects critical assets below a safe score.
- The adaptive baseline **freezes during inflammation** (an attack window), so the attacker's behaviour is never absorbed as "normal."

- **Beaconing** (regular phone-home) and **lateral-movement fan-out** (one account, many hosts) are detected.
- The **attack-memory lookup** and **incident timeline** bind evidence correctly, and the **Bayesian hypothesis update** combines signature and anomaly evidence.
- **Graduated-response bands** map a posterior to the right action level.
- **Quarantine auto-lifts** — a full apply → tick → lift cycle is exercised, proving containment is never permanent.
- The **alert-fatigue multiplier** rises with the false-positive rate, and the **exhaustion budget** bounds rule evaluation under load.

The interface invariant

A dedicated test, `processors_allInterfaceTyped`, asserts that **every** signal processor depends on an *interface*, never a concrete neuron class. This is what makes detectors swappable per deployment without touching the rest of the system — a structural quality, automatically enforced. The full worker suite passes with the security module added, with **no regressions**.

6 Layer 2 — Behavioural tests (the full-run demo)

The full-run demo runs the three realistic streams through the real worker entry path and checks seven assertions. All seven hold:

Assertion	Plain meaning	Result
temporalAttackChainDetected	The ordered attack chain was raised	Pass
maintenanceContextSuppressed	Planned maintenance was calmed, not alarmed	Pass
lowAndSlowCorrelation	The quiet data leak was surfaced	Pass
baselineFrozenDuringAttack	Learning froze during the attack window	Pass
allTrainingSourcesReferenced	Every training source family is represented	Pass
attackScoreGreaterThanBenign	The attack scored higher than benign maintenance	Pass
advisoryOnly	No active blocking action was ever emitted	Pass

Together these assertions prove the property that single-event tools cannot deliver: the system raises real attacks, calms benign context **without erasing its evidence**, catches the low-and-slow case, and stays strictly advisory throughout.

7 Layer 3 — Trained-model quality metrics

The trained correlator was evaluated on held-out windows, split by campaign / host / time / attack type so the test measures genuinely unseen patterns. The results:

Split	Windows	True pos.	False pos.	Precision	Recall	F1	FP rate
Training	10,000	7,000	0	1.0	1.0	1.0	0.0

Validation	6,000	4,000	0	1.0	1.0	1.0	0.0
Test	5,000	3,000	0	1.0	1.0	1.0	0.0
Overall	21,000	14,000	0	1.0	1.0	1.0	0.0

Mean time to detection is effectively immediate: **0.126 ticks** on training, **0.0** on validation and test, **0.063** overall. In plain terms, when the model does detect an attack chain on this data, it does so at the first decisive evidence rather than after the damage is done.

> **Read these numbers correctly**
>
> Zero false positives and perfect recall on a clean, deterministic reference corpus is exactly what a correctly wired pipeline should produce — the reference data is separable by design. This confirms the machinery; it does not forecast performance on noisy real-world telemetry. See Section 11.

8 Calibration: are the probabilities honest?

A confidence score is only useful if it means what it says — a window scored 0.9 should really be an attack about 90% of the time. This is called **calibration**. On the reference corpus the model's scores are sharply, correctly separated:

Score band	Windows	Real attack rate	Mean score
0.0 – 0.1 (confident benign)	7,000	0% — none were attacks	0.004
0.9 – 1.0 (confident attack)	14,000	100% — all were attacks	0.995

The model places benign windows near 0 and attack windows near 1, with almost nothing in the uncertain middle. Higher score reliably means higher real risk — the monotonic ordering the production gate requires.

9 What the model learned (feature evidence)

Because the model is deliberately transparent, we can read *why* it decides as it does. The evidence it weighs most heavily tells a coherent, sensible story — it is not a black box.

Evidence that pushes toward "attack"

Feature	Weight	Plain meaning
technique_command_and_control	+0.43	A machine appears to be phoning home to an attacker
network_receptor_score	+0.42	Strong network-side evidence
max_threat_intel	+0.42	A high-confidence external threat indicator
slow_context_score	+0.39	Threat intel and asset criticality combined
mean_evidence	+0.39	Consistently strong evidence across the window

Evidence that pushes toward "benign"

Feature	Weight	Plain meaning
maintenance_ratio	-0.45	It happened during approved maintenance
benign_context_ratio	-0.39	Mostly expected, benign context
technique_unusual_login	-0.27	An isolated odd login with nothing following it
source_diversity	-0.18	Evidence from one source only, not a coordinated chain

Note how the strongest *calming* signal is approved maintenance — the model has learned the single most common cause of false alarms and weighs it down, exactly as a careful analyst would.

10 Layer 4 — Production-scale pipeline evidence

The production-scale run records a 100 GiB *logical* corpus target by deterministically replicating the reference campaigns, without writing 100 GiB to disk. This stress-tests the scaling metadata and guardrails on an ordinary workstation.

Measure	Value
Fitted reference multiplier	1,000
Fitted events / windows	96,000 / 21,000
Estimated fitted corpus	41.45 MiB
Effective reference multiplier	2,411,186
Effective events	231,473,856
Effective temporal windows	50,634,906
Target corpus size	100.00 GiB
Target reach ratio	0.99997 (99.997%)
Training epochs	2,400 (cap 4,096 windows/epoch)

Verification performed on this run: the trainer's Python compiled cleanly; the pipeline completed with test F1 above the 0.85 gate; the JSON artifacts validated the 100 GiB target, the fitted and effective counts, and the held-out false-positive rate; and — importantly — **no 100 GiB file was written**, so the repository is never inflated by generated data.

The run also produced and validated the **deployable JNeopallium network** the worker boots from: five layer files plus `model-descriptor.json` describing 5 layers, 8 real neurons, 34 trainable weights, and 1 bias, each neuron referencing a concrete runtime security class. The trained weights, decision threshold (0.2), sequence gates, and the baseline-freeze policy (freeze at posterior 0.3 or signature confidence 0.8) are embedded directly in the correlation layer — so **the artifact that was tested is the artifact that deploys**, with no translation step in between.

11 The honest limitations (read this carefully)

We hold ourselves to the same standard we ask of the model: tell the truth about the evidence. Three limitations bound everything above.

1. **The reference corpus is synthetic and deterministic.** It is built to be separable so the pipeline can be exercised and regression-tested repeatably. Perfect scores on it confirm correctness of the machinery, not real-world detection quality.
2. **Real accuracy must be earned on external data.** Genuine production claims require LANL, ToN_IoT, OpTC, CIC/CSE-CIC, UNSW-NB15, and controlled CALDERA campaigns — with calibration curves, per-entity false-positive budgets, and streaming-memory/backpressure tests on representative enterprise telemetry.
3. **Advisory-only by default.** Active enforcement (blocking, host isolation) is deliberately out of scope here and requires a separate safety case, approval workflow, and rollback path before it may be enabled.

> **Why we lead with the caveat instead of hiding it**
>
> A security vendor that buries the difference between pipeline evidence and real-world accuracy is a vendor you cannot trust with your network. The honesty is the product feature: every advisory carries its evidence, and every claim in this report carries its scope.

12 Overall verdict

The cybersecurity module passes every test at every level on the reference corpus. The components behave correctly and are structurally swappable; the running system makes the right calls on three realistic streams and stays strictly advisory; the trained model separates attack from benign with honest, well-calibrated confidence; and the training pipeline is correct, bounded, and reproducible at a 100 GiB logical scale.

The system is **ready for the next stage**: external-dataset validation on representative telemetry, which is the only thing that can convert this strong pipeline evidence into a real-world accuracy claim. The architecture, the safety design, and the evidence discipline needed for that stage are already in place.

13 Glossary

Term	Meaning
Window	A slice of time on one entity that the model judges as attack or benign
True / false positive	A correct / incorrect "this is an attack" judgement
Precision / recall	Share of flags that were right / share of real attacks that were caught
F1	A single fair blend of precision and recall
Calibration	Whether a confidence score means what it says
Tick	The platform's internal unit of time
Posterior	The model's probability that an attack is underway

Regression	A change that quietly breaks behaviour that used to work
Deterministic	Same input and command always produce the same output

Jneopallium Cybersecurity Module · Demo 06 · Complete Test Report. Results are pipeline evidence on a deterministic reference corpus, not a real-world accuracy claim. Safety mode: ADVISORY. License: BSD 3-Clause.